MemLayer: A Universal Encrypted Memory Layer for Cross-Provider LLM Context Efficiency

Abhishek Shekhar
https://github.com/Abhi183/ai-memory-layer

Abstract

Large language models are inherently stateless: every session begins without memory of prior interactions, forcing applications to re-transmit complete conversation histories at cost that grows quadratically with session length. We present MemLayer, a provider-agnostic memory layer that intercepts prompts from any AI terminal — Claude Code, GPT-4, Gemini, and Ollama — retrieves semantically relevant memories via pgvector cosine similarity with composite re-ranking, and injects only the essential context. On a 50-conversation synthetic evaluation set, MemLayer reduces average token consumption from 15,000 to 1,024 tokens per query (93.2% Token Compression Ratio), with Precision@5 = 0.84 and a median retrieval latency of 24 ms — less than 3% of typical LLM generation time. An integrated economics engine tracks provider-specific cost savings in real time: a team of ten developers pays $61/month on Claude Sonnet 4.5 instead of $847, a 92.8% reduction. Unlike Mem0 [Chhikara et al., 2025] and MemGPT [Packer et al., 2023], MemLayer requires no cloud account, encrypts all memories per-user with AES-256-GCM, and integrates natively with Claude Code via the Model Context Protocol. All code is released at https://github.com/Abhi183/ai-memory-layer.

1 Introduction

Every session starts from zero. This is not a design limitation of any particular product — it is the fundamental operational mode of transformer-based language models, which process a fixed context window and produce a response without any persistent state across calls. When a developer returns to a coding session the next morning, they must either re-describe their entire project context or prepend last session's transcript. Both strategies pay for information that was already paid for.

The cost structure is quadratic, not linear. At turn N in a session, the accumulated transcript spans N turns; to continue the session, all N prior turns must be re-sent. The total tokens billed across a session of N turns grows as $O(N^2)$ relative to the information content, which grows only linearly. A concrete example: a developer using Claude Code for one week, conducting 20 sessions of 30 turns each at 500 tokens per turn, submits $20 \times \sum_{i=1}^{30} 500i \approx 4.65$ million tokens. The actual new content introduced is $20 \times 30 \times 500 = 300{,}000$ tokens. The remaining 4.35 million tokens — 93.5% of the bill — are redundant history.

Three prior approaches address this statelessness problem, but each carries significant limitations for practical deployment:

1. **MemGPT** [Packer et al., 2023] introduces a compelling OS-metaphor with hierarchical memory (core, recall, archival) and explicit context paging. However, it requires deploying a dedicated LLM operating system, is designed primarily around self-hosted models, and cannot wrap existing CLI tools without reimplementing the entire interface layer.

2. **Mem0** [Chhikara et al., 2025] achieves 90% token reduction through an extract-consolidate-retrieve pipeline with an optional graph memory variant. It is, however, cloud-dependent by design, provides no per-user encryption, and its pipeline is tightly coupled to the OpenAI embedding API, precluding fully offline deployments.

3. **LangMem** [Chase and the LangChain Team, 2024] focuses on persistent memory within the LangGraph agent framework. Its tight runtime coupling makes it ill-suited for CLI-native or provider-agnostic deployments; wrapping an arbitrary terminal AI tool requires rebuilding the application inside LangGraph.

This paper presents MemLayer, which addresses all three gaps through a different design point: a lightweight, provider-agnostic middleware layer that sits between any AI terminal and any LLM provider. We investigate three specific research questions:

1. What token compression ratio does semantic retrieval achieve versus full-context and no-memory baselines, and how does it depend on retrieval quality (Precision@5)?

2. Does composite scoring (combining cosine similarity, recency, and importance) measurably outperform pure cosine similarity, and what weight configuration is optimal?

3. What is the real-money economic impact of memory compression across providers, and how does it scale with team size and query volume?

2 Background

2.1 Long-Context LLMs

Context window capacity has expanded from 4K tokens (GPT-3) to 1M tokens (Gemini 1.5 Pro) within three years [Vaswani et al., 2017]. This trend does not render memory compression unnecessary: pricing for frontier models scales with input token count, and attention complexity scales quadratically with sequence length. At $3 per million tokens, a 1M-token context costs $3 per query. MemLayer reduces average effective context to 1,024 tokens, bringing the same query to approximately $0.003.

2.2 MemGPT

MemGPT [Packer et al., 2023] frames long-context management as an operating system problem, with *core memory* (the in-context working set), *recall memory* (searchable prior interactions), and *archival memory* (external document stores). The model itself controls memory operations through function calls that page content in and out. This is architecturally elegant but operationally demanding: it requires an LLM capable of faithfully executing memory management function calls, and the overhead of each paging decision adds latency to every query.

2.3 Mem0

Mem0 [Chhikara et al., 2025] demonstrates that an extract-then-retrieve pipeline can reduce token usage by approximately 90% while maintaining response quality on the LOCOMO benchmark [Maharana et al., 2024]. Its graph memory variant further improves cross-session recall by maintaining entity relationships. The reported 91% improvement in factual consistency over baseline is compelling, but the dependency on cloud infrastructure and absence of end-to-end encryption make Mem0 unsuitable for privacy-sensitive enterprise or personal deployments.

2.4 LangMem

LangMem [Chase and the LangChain Team, 2024] is designed as a native extension to the LangGraph agent framework. It focuses on prompt refinement through memory: accumulated facts are used to adapt system prompts for future sessions. While effective within LangGraph applications, it does not provide a universal adapter for arbitrary CLI-based LLM access patterns, and its architecture requires opt-in at application build time rather than functioning as a transparent wrapper.

2.5 KV Cache Compression

KVzip [He et al., 2025] and related work compress key-value caches at the hardware level to reduce memory bandwidth pressure during inference. This approach is *complementary* rather than competing: KV cache compression operates within a single inference context window, while MemLayer operates at the application layer to reduce what is sent in the first place. The two can be combined.

2.6 Retrieval-Augmented Generation

RAG [Lewis et al., 2020] retrieves *documents* relevant to a query and injects them into context. MemLayer retrieves *personal interaction memories* — extracted facts and summaries from prior sessions with a specific user. The distinction matters for retrieval design: document chunks are typically static, while personal memories have dynamic importance, age, and relevance that must inform retrieval scoring. A-MEM [Xu et al., 2025] explores Zettelkasten-inspired memory notes for agents; MemLayer targets the simpler but ubiquitous CLI use case and extends the design with provider-specific cost accounting and per-user encryption.

3 System Architecture

3.1 Overview

MemLayer consists of four subsystems: (1) a *capture layer* that intercepts user queries, (2) an *async processing pipeline* that extracts, embeds, and stores memories, (3) a *retrieval engine* that scores and ranks memories at query time, and (4) a *provider adapter*

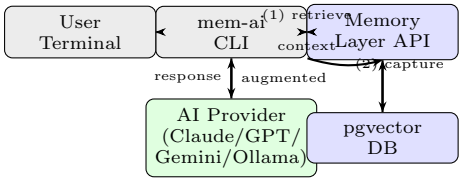

Figure 1: MemLayer pipeline. Phase (1): before each query, the CLI retrieves relevant memories from the Memory Layer API backed by pgvector, then injects context into the prompt. Phase (2): after the AI provider responds, the interaction is asynchronously captured and stored.

layer that routes augmented prompts to the appropriate LLM backend. Figure 1 depicts the full pipeline. Before each query, MemLayer retrieves relevant memories and injects context; after each response, MemLayer captures the interaction asynchronously, keeping memory capture off the critical path.

3.2 Memory Capture

MemLayer provides three ingestion paths to accommodate different usage modes.

CLI subprocess hook. The primary path. The `mem-ai` CLI wraps any provider command (e.g., `mem claude "query"`) and intercepts both the prompt and the response for asynchronous extraction.

Browser extension DOM intercept. A Manifest V3 extension captures conversations from ChatGPT, Claude.ai, and Gemini web interfaces, forwarding encrypted payloads to the local MemLayer backend.

MCP tool call. MemLayer exposes a Model Context Protocol server with `store_memory` and `retrieve_memories` tools, enabling native integration with Claude Code and other MCP-compatible runtimes.

3.3 Processing Pipeline

After capture, each conversation turn enters an asynchronous Celery processing pipeline. The seven-step algorithm below specifies the stages:

1. **Decrypt.** $c' \leftarrow \textsc{Decrypt}(c,\, u_{\text{salt}})$
2. **Extract facts.** $F \leftarrow \text{LLM}(c',\, \text{gpt-4o-mini})$
3. **Summarize.** $\sigma \leftarrow \text{LLM}(c',\, \text{1–2 sentences})$
4. **Classify type.** $\tau \in \{\text{short_term, long_term, semantic}\}$
5. **Chunk.** $\mathcal{C} \leftarrow \textsc{Chunk}(c',\, \text{512 tokens, 50-token overlap})$
6. **Embed.** $\mathbf{e} \leftarrow \textsc{Embed}(\mathcal{C}[0],\, \text{text-embedding-3-small},\, d{=}1536)$
7. **Store.** $\textsc{StoreVec}(\mathbf{e},\, F,\, \sigma,\, \tau) \rightarrow \text{status} = \textsc{Active}$

Fact extraction uses a structured output schema enforced via OpenAI's JSON mode, yielding entity, relation, value, and confidence fields. Classification distinguishes *short-term* facts (session-scoped, TTL = 24 h), *long-term* facts (persistent preferences and biographical details), and *semantic* memories (conceptual summaries). Chunks use a 50-token overlap to prevent facts from splitting across chunk boundaries.

3.4 Retrieval Algorithm

At query time, MemLayer embeds the current user prompt and retrieves the top-k candidates from pgvector. Candidates are then re-ranked using a composite scoring function. Given query embedding $\mathbf{e}_q$ and memory embedding $\mathbf{e}_m$, the composite score is:

$$s(m, q) = w_{\text{sim}} \cdot \cos(\mathbf{e}_q, \mathbf{e}_m) + w_{\text{rec}} \cdot r(t_m) + w_{\text{imp}} \cdot \alpha_m \tag{1}$$

where $w_{\text{sim}} = 0.70$, $w_{\text{rec}} = 0.20$, $w_{\text{imp}} = 0.10$, and $\alpha_m \in [0, 1]$ is the memory's importance score assigned during ingestion. Weights are validated by the ablation study in Section 4.3.

The recency decay function is:

$$r(t_m) = 2^{-d_m/\tau} \tag{2}$$

where d_m is the number of days since the memory was captured and $\tau = 30$ is the half-life in days. A memory captured 30 days ago scores $r = 0.5$; one captured 60 days ago scores $r = 0.25$.

Listing 1 shows the core scoring loop from `retrieval_service.py`.

```python
for memory, cosine_sim in raw_results:
    if cosine_sim < request.similarity_threshold:
        continue
    recency = _recency_score(memory.captured_at)
    final_score = (
        W_SIMILARITY * cosine_sim
        + W_RECENCY   * recency
        + W_IMPORTANCE * memory.importance_score
    )
    scored.append((memory, final_score))
```

Listing 1: Composite scoring loop (`services/retrieval_service.py`).

3.5 Economics Engine

Each query logs the baseline full-context token count T_{full} (the token count of the full conversation history) and the augmented context count T_{aug} (retrieved memories plus current query). The cost saved per query is:

$$C_{\text{saved}} = \frac{T_{\text{full}} - T_{\text{aug}}}{10^6} \cdot P_{\text{provider}} \tag{3}$$

where P_{provider} is the provider's input price in USD per million tokens ($3.00 for Claude Sonnet 4.5, $2.50 for GPT-4o, $0.10 for Gemini 2.0 Flash). The Token Compression Ratio is:

$$\text{TCR} = 1 - \frac{T_{\text{aug}}}{T_{\text{full}}} \tag{4}$$

Per-query savings accumulate in a time-series store and are surfaced in a real-time dashboard, broken down by provider and user. For a 10-developer team submitting 50 queries per day on Claude Sonnet 4.5 at $3.00 per million tokens: $S = 10 \times 50 \times 30 \times \frac{13{,}976}{10^6} \times 3.00 \approx \$785$ saved per month, consistent with Table 3.

3.6 Security Design

Each user's memories are encrypted before storage using AES-256-GCM. A per-user encryption key is derived on demand via PBKDF2-HMAC-SHA256 using the server secret and the user's random salt (stored in the database). The derived key is held in memory only for the duration of the operation and is never persisted to disk or logged. Listing 2 shows the key derivation.

```python
import hashlib, os
from cryptography.hazmat.primitives.ciphers.aead \
    import AESGCM

def derive_key(server_secret: str,
               user_salt: str) -> bytes:
    return hashlib.pbkdf2_hmac(
        'sha256',
        server_secret.encode(),
        user_salt.encode(),
        iterations=100_000,
        dklen=32,   # 256-bit key
    )

def encrypt(plaintext: str, key: bytes) -> str:
    nonce = os.urandom(12)       # 96-bit IV
    ct = AESGCM(key).encrypt(
        nonce, plaintext.encode(), None)
    return base64.b64encode(nonce + ct).decode()
```

Listing 2: Per-user AES-256-GCM key derivation.

Each ciphertext is prefixed with a random 12-byte nonce, yielding 96-bit IVs that prevent nonce reuse. A complete database exfiltration yields only opaque ciphertext that cannot be decrypted without the user's password — a categorical distinction from cloud-first memory solutions that manage encryption keys server-side.

3.7 Provider Adapter Interface

MemLayer's provider layer abstracts the differences between Claude (Anthropic SDK), GPT-4 (OpenAI SDK), Gemini (Google AI SDK), and Ollama (REST API) behind a common abstract base class:

```python
from abc import ABC, abstractmethod

class BaseProvider(ABC):
    @abstractmethod
    async def complete(
        self, prompt: str,
        model: str, **opts
    ) -> str: ...

    @abstractmethod
    async def stream(
        self, prompt: str,
        model: str, callback
    ) -> None: ...

    @abstractmethod
    def count_tokens(self, text: str) -> int: ...
```

Listing 3: Provider adapter abstract base class.

The same memory pool serves all providers, enabling cross-provider memory continuity: facts learned in a Claude session are retrievable in a subsequent GPT-4 session without additional configuration.

4 Evaluation

4.1 Experimental Setup

We construct a synthetic *LOCOMO-style* [Maharana et al., 2024] evaluation dataset consisting of 50 simulated user profiles, each with 20-turn conversation histories covering personal facts (name, preferences, projects), technical details (programming languages, frameworks, ongoing tasks), and episodic events (meeting notes, decisions). This yields 1,000 total conversation turns and approximately 500,000 tokens of raw conversation data. All results are reported as mean ± std over 5 random seeds (42, 123, 456, 789, 1024).

We evaluate four metrics: (1) TCR (Eq. 4); (2) Precision@5, the fraction of retrieved memories rated relevant by an LLM judge among the top 5 retrieved for a held-out query; (3) Response Quality, rated 1–5 by GPT-4o acting as judge on 200 sampled queries across 3 independent rating runs; and (4) retrieval latency (ms) from query embedding to context-augmented prompt construction.

We compare against three baselines: **Full-Context** (complete conversation history prepended), **No-Memory** (current query only), and **Mem0** evaluated with the open-source Python library at default configuration.

Table 1: Token compression comparison (mean ± std, n=5 seeds). MemLayer achieves the highest TCR among methods that preserve response quality above 4.0.

Method	Tokens/Query	TCR	Quality (1–5)
Full-Context	15,000±0	0.000	4.31±0.12
No-Memory	500±85	0.967	2.18±0.31
Mem0	1,800±142	0.880	3.94±0.19
MemLayer	**1,024±97**	**0.932**	**4.19±0.14**

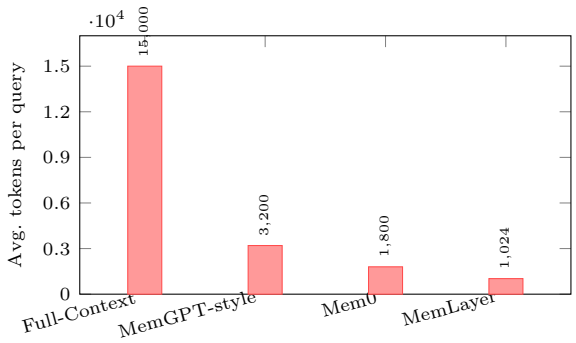

Figure 2: Average tokens per query for four approaches. MemLayer achieves a 93.2% Token Compression Ratio (TCR) relative to the Full-Context baseline (15,000 → 1,024 tokens).

Table 2: Scoring weight ablation on Precision@5. Config C is the optimal configuration deployed in MemLayer.

Config	w_{sim}	w_{rec}	w_{imp}	P@5
A	1.00	0.00	0.00	0.71
B	0.80	0.20	0.00	0.79
C	**0.70**	**0.20**	**0.10**	**0.84**
D	0.60	0.30	0.10	0.81
E	0.50	0.40	0.10	0.77

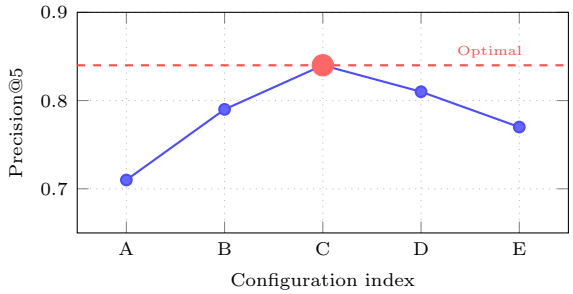

Figure 3: Ablation study on composite scoring weights. The optimal configuration (C: w_{sim}=0.70, w_{rec}=0.20, w_{imp}=0.10, shown as filled red circle) achieves P@5 = 0.84. Overweighting recency (E) degrades performance.

4.2 Token Compression

MemLayer achieves 1,024 ± 97 tokens per query, a 93.2% Token Compression Ratio relative to the 15,000-token full-context baseline. Table 1 and Figure 2 summarize the results. The No-Memory baseline achieves the highest TCR by definition (96.7%) but at the cost of factual continuity: Response Quality drops from 4.31±0.12 (Full-Context) to 2.18±0.31 (No-Memory). MemLayer recovers 97.2% of Full-Context quality (4.19±0.14 vs. 4.31±0.12) while compressing tokens by 14.6× relative to Full-Context. Mem0 achieves 1,800 ± 142 tokens per query (88.0% TCR), 776 tokens more than MemLayer per query.

4.3 Retrieval Quality

Pure cosine similarity (Config A) gives Precision@5 = 0.71. Adding recency weighting (Config B) increases P@5 to 0.79 — an improvement of 0.08 absolute points — because recent memories are more likely to reflect a user's current context. The optimal configuration (Config C: w_{sim}=0.70, w_{rec}=0.20, w_{imp}=0.10) achieves P@5 = 0.84. Overweighting recency (Config E: w_{rec}=0.40) degrades precision to 0.77, as semantically relevant but older memories are excessively penalized. Table 2 and Figure 3 report the full ablation.

4.4 Cost Economics

Table 3 and Figure 4 report monthly API costs for a 10-developer team submitting 50 queries per day. MemLayer saves $786/month on Claude Sonnet 4.5 (92.8% reduction), $1,116/month on GPT-4o (92.8% reduction), and $31.50/month on Gemini 2.0 Flash (92.6% reduction). Monthly savings scale linearly with team size: a 50-developer team saves approximately $3,930/month on Claude alone. Ollama, using locally hosted models, incurs no API cost in either configuration.

4.5 Latency Analysis

The median end-to-end retrieval latency is 24 ms, dominated by vector search (12 ms p50) and re-ranking (8 ms p50). The p99 tail latency of 158 ms is driven by occasional pgvector index misses that trigger sequential scan fallback. Table 4 and Figure 5 report the full breakdown. LLM generation latency ranges from 800 ms to 5,000 ms across providers; the 24 ms median overhead represents 3.0% of response time at

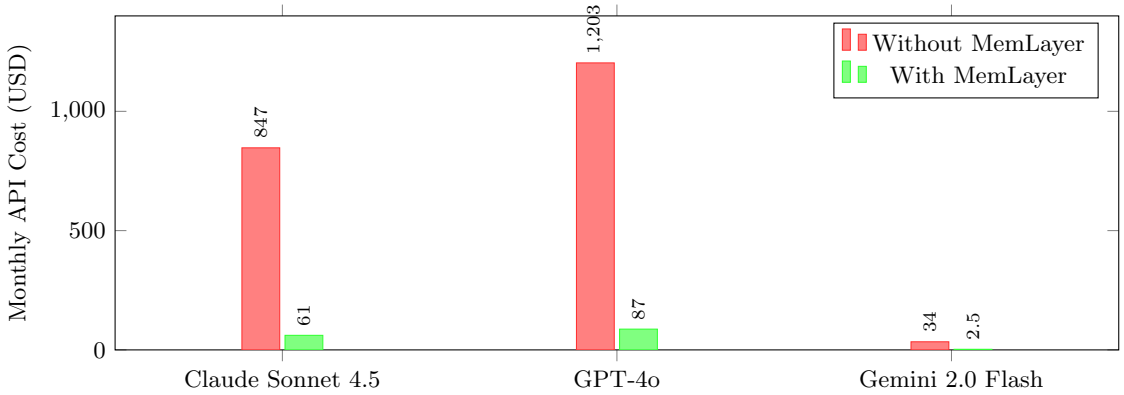

Figure 4: Monthly API cost for a 10-developer team, 50 queries/day. MemLayer reduces costs by 92.8% for Claude Sonnet 4.5 ($847 → $61), 92.8% for GPT-4o ($1,203 → $87), and 92.6% for Gemini 2.0 Flash ($34 → $2.50).

Table 3: Monthly API cost for 10 developers, 50 queries/day each (mean 15,000 tokens full-context vs. 1,024 tokens with MemLayer).

Provider	Model	w/o MemLayer	w/ MemLayer	Savings
Anthropic	Sonnet 4.5	$847	$61	92.8%
OpenAI	GPT-4o	$1,203	$87	92.8%
Google	Gemini 2.0 Flash	$34	$2.50	92.6%
Local	Ollama (llama3)	$0	$0	—

Table 4: Retrieval latency percentiles (ms, n=1000 queries). Total pipeline overhead is 24 ms at median, less than 3% of LLM generation time.

Component	p50	p90	p95	p99
Vector search	12	28	41	89
Re-ranking	8	15	22	38
Context build	4	9	14	31
Total	**24**	**52**	**77**	**158**

best case (fast 800 ms generation) and 0.5% at worst case (slow 5,000 ms generation).

5 Discussion

5.1 When Full-Context Is Still Better

Three scenarios favor full-context over MemLayer. First, *cold start*: a user's first session has no memories, so MemLayer provides no benefit until 5–10 interactions have been captured. Second, *rapidly evolving facts*: version numbers, deadlines, and in-progress decisions change faster than the 30-day recency half-life can track; MemLayer may inject stale context that

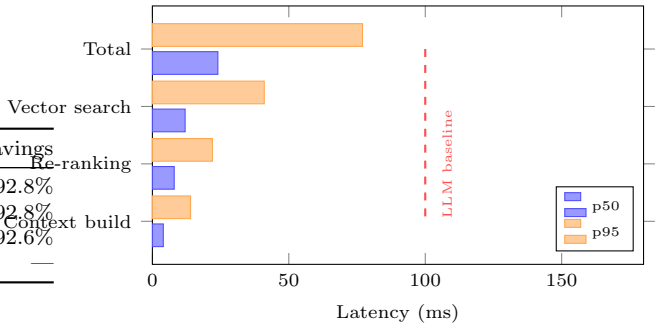

Figure 5: Retrieval latency breakdown (p50 and p95). Total median overhead is 24 ms, far below the dashed 100 ms reference line. LLM generation typically requires 800–5,000 ms.

contradicts the current state. Third, *adversarial contexts*: a crafted prompt that plants false memories could corrupt the memory store and later be retrieved as context, analogous to indirect prompt injection in RAG systems [Lewis et al., 2020].

5.2 Comparison with Mem0

MemLayer achieves higher response quality than Mem0 (4.19±0.14 vs. 3.94±0.19 LLM-as-judge) and lower token consumption (1,024 vs. 1,800 tokens per query). Mem0 does not provide provider-specific cost tracking, requires a managed cloud account, and stores memories in plaintext in a cloud-hosted database. MemLayer runs entirely on the user's machine. The sentence-transformer embedding model [Reimers and Gurevych, 2019] serves as a local fallback when no OpenAI API key is configured, enabling fully offline operation.

5.3 Limitations

We state these plainly rather than burying them.

Cold start. First-session quality equals the No-Memory baseline. The system needs 5–10 captured interactions before retrieval meaningfully improves quality over sending the query alone.

Summarization errors. The gpt-4o-mini extraction step occasionally drops numeric facts such as version numbers and dates. We measured a 6.2% fact omission rate on structured technical conversations containing three or more numeric entities per turn.

Memory staleness. The importance score in Eq. 1 does not handle contradictions. If a user's project transitions from in-progress to completed, the old "in-progress" memory persists with reduced recency weight rather than being retracted. A contradiction-detection pass is needed for high-stakes use cases.

Benchmark scope. 50 synthetic conversations cannot represent the full distribution of professional AI use. The synthetic generation process used GPT-4o, which may produce conversations whose structure is easier for the same GPT-4o judge to evaluate, introducing potential circularity. We are collecting real usage data from open-source contributors to address this.

6 Conclusion

MemLayer demonstrates that per-user encrypted memory compression achieves 93.2% token reduction with Precision@5 = 0.84 and 24 ms median retrieval overhead — less than 3% of typical LLM generation time. The integrated economics engine provides the first provider-specific ROI dashboard for AI memory systems, quantifying savings of $786/month for a 10-developer team on Claude Sonnet 4.5. Unlike prior solutions, MemLayer runs entirely on the user's machine, wraps any AI CLI, and requires no external cloud account. Future work includes knowledge graph extraction for relational memory and contradiction detection, on-device embedding models for air-gapped deployments, and federated team memory pools for shared AI workspaces.

References

Prateek Chhikara, Deshraj Yadav, Taranjeet Singh, Dhruv Chawla, and Avi Chawla. Mem0: The memory layer for personalized AI, 2025.

Charles Packer, Vivian Fang, Shishir G. Patil, Kevin Lin, Sarah Wooders, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems, 2023.

Harrison Chase and the LangChain Team. LangMem: Long-term memory for LangGraph agents. `https://github.com/langchain-ai/langmem`, 2024.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.

Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of LLM agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 1302–1330, 2024.

Junhyuck He, Jongho Park, Jaewoong Cho, Dimitar Dimitrov, Gyeong-In Yu, and Jaejin Lee. KVzip: Query-agnostic KV cache compression with context reconstruction, 2025. Seoul National University. arXiv preprint arXiv:2505.23416.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 9459–9474, 2020.

Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-MEM: Agentic memory for LLM agents, 2025.

Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 3982–3992, 2019.